



Resource Sharing



Luís Miguel Pinho

Resource Sharing

- ▶ What about resource sharing?
 - ▶ Let's revisit the model
 - ▶ Set of n dependent tasks $(\tau_1, \tau_2, \tau_3, \dots, \tau_n)$.
 - ▶ Most systems, tasks share resources apart the processor
 - ▶ Memory areas
 - ▶ Network
 - ▶ ...
- ▶ Synchronisation mechanisms such as locks are used to manage shared resources
 - ▶ When a task tries to access a resource locked by another task, it blocks. When the resource is again available, the blocked task becomes ready for execution.

► Example

Resource Sharing

▶ Priority Inversion

- ▶ In pre-emptive systems, in principle it should execute the task with the highest priority.
- ▶ However, due to waiting in the access to locked resources, a high-priority task may need to wait for low-priority tasks having the lock on the resource (and others of intermediate priority)
- ▶ This is a priority inversion

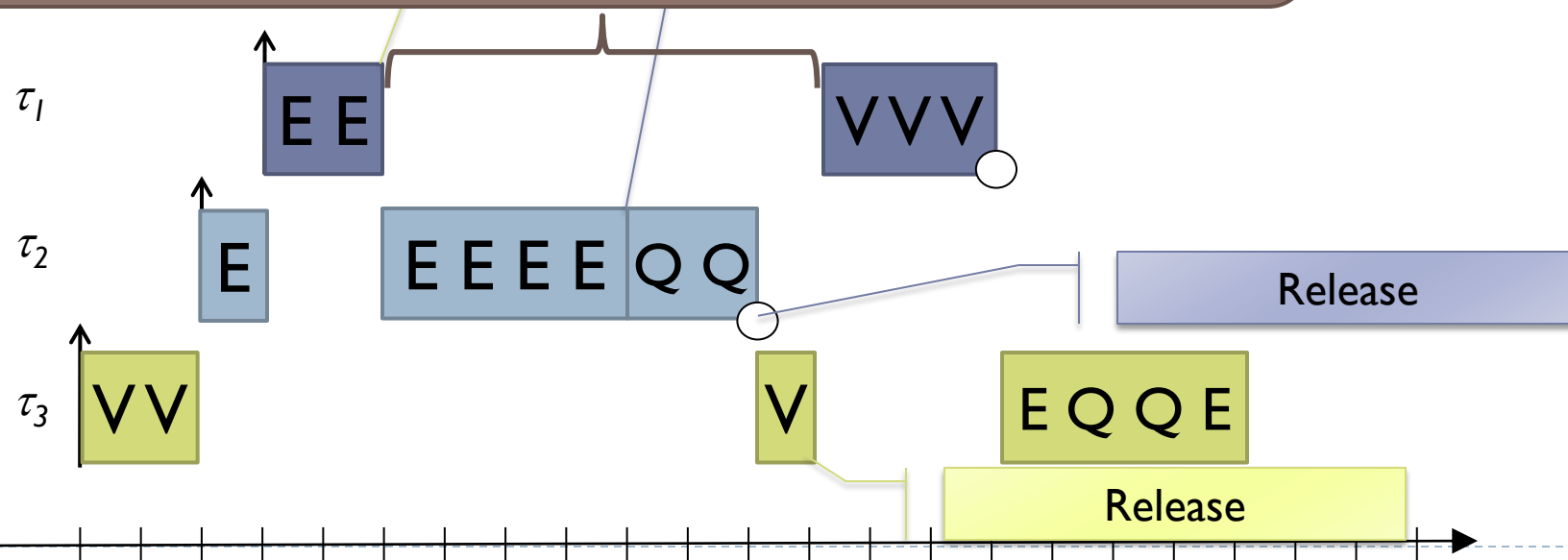
Resource Sharing

► Priority Inversion

	C (ms)	$T=D$ (ms)	Priority	Arrival instant (ms)	Execution sequence
τ_1	5	17	3	3	EEVVV
τ_2	5	15	2	2	EEEEQQ
τ_3	7	35	1	0	VVVEQQE

Priority Inversion

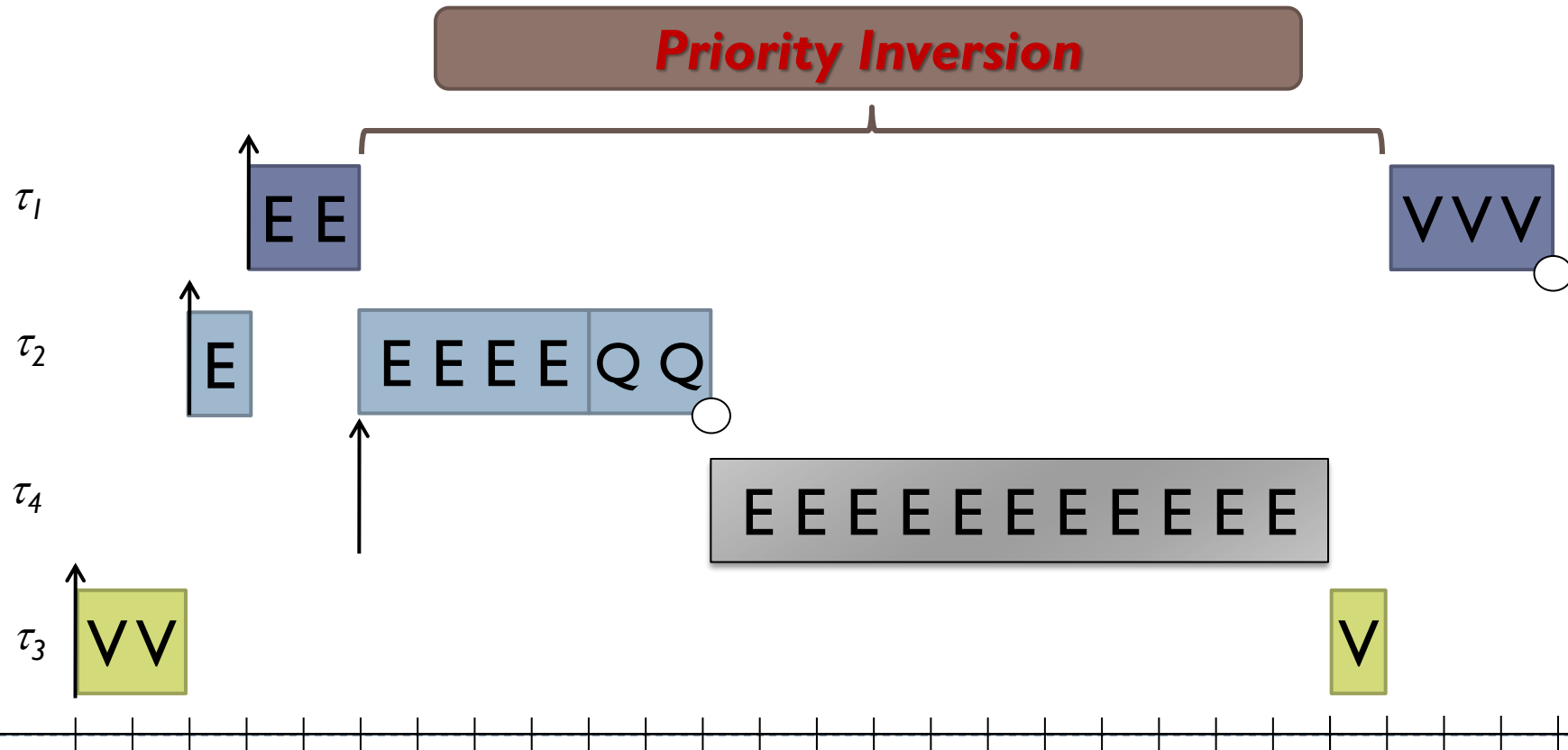
A higher priority task (τ_1) waits for a lower priority task (τ_3). And middle periodic tasks are allowed to execute.



Resource Sharing

► Priority Inversion

- If no rule is applied in sharing resources, it is not possible to determine maximum blocking time (or the blocking time is large)



Priority Inversion

- ▶ Avoiding priority inversion
 - ▶ Priority inversion is inevitable in the presence of shared resources usage (intrinsic to blocking)
 - ▶ However, we need to limit and be able to quantify the impact in the worst-case response time to guarantee schedulability of the task set
- ▶ To address this problem several protocols are used to manage shared resources
 - ▶ These protocols are used to grant exclusive access to a shared resource, but bounding the period of priority inversion, and being analyzable, i.e., allowing to determine the worst-case blocking that a task can have

Priority Inversion

- ▶ Avoiding priority inversion

- ▶ Inhibiting interrupts

- ▶ When using a shared resource, no interruptions

- Easy to implement, but blocks everything else, including timers – should be used only for very short critical regions
 - Each task can only be blocked once by lower priority (or further deadline) tasks, and only for the duration of the (hopefully small) critical region

- ▶ Preemption inhibiting

- ▶ When using a shared resource all other tasks are blocked, but not interrupt service routines (tick, timers, etc).
 - ▶ However, contrarily to disabling the interrupts, in this case the interrupt service routines,

- ▶ High-level protocols

- ▶ Goal is to only block tasks which share resources
 - ▶ If possible also prevent deadlocks

Priority Inversion

▶ Avoiding priority inversion

▶ Most important protocols

▶ *PIP - Priority Inheritance Protocol*

- *Allows several blocking periods but guarantees maximum blocking period*

▶ *PCP - Priority Ceiling Protocol*

- *Allows only one blocking period but introduces unnecessary blocking*

▶ *IPCP - Immediate Priority Ceiling Protocol*

- *Also known as PCE - Priority Ceiling Emulation*
- *Same as before, but simpler to implement*
- *Also prevents deadlocks*

▶ *SRP – Stack Resource Policy*

- *Similar to IPCP but allows for dynamic priorities*

▶ *DFP – Deadline Floor Protocol*

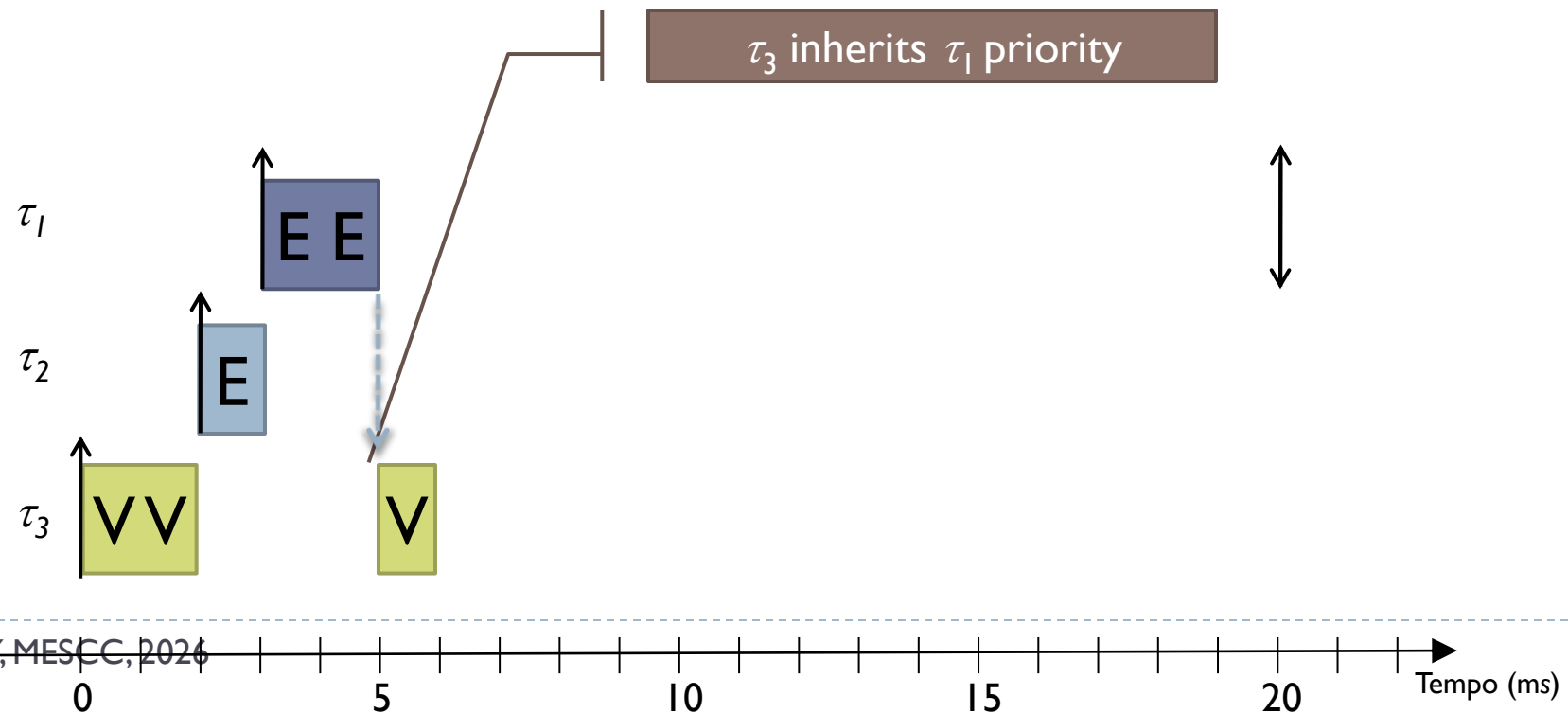
- *Simplification of SRP*

▶ We will focus on the first and three last ones

Resource Sharing

► Priority Inheritance Protocol

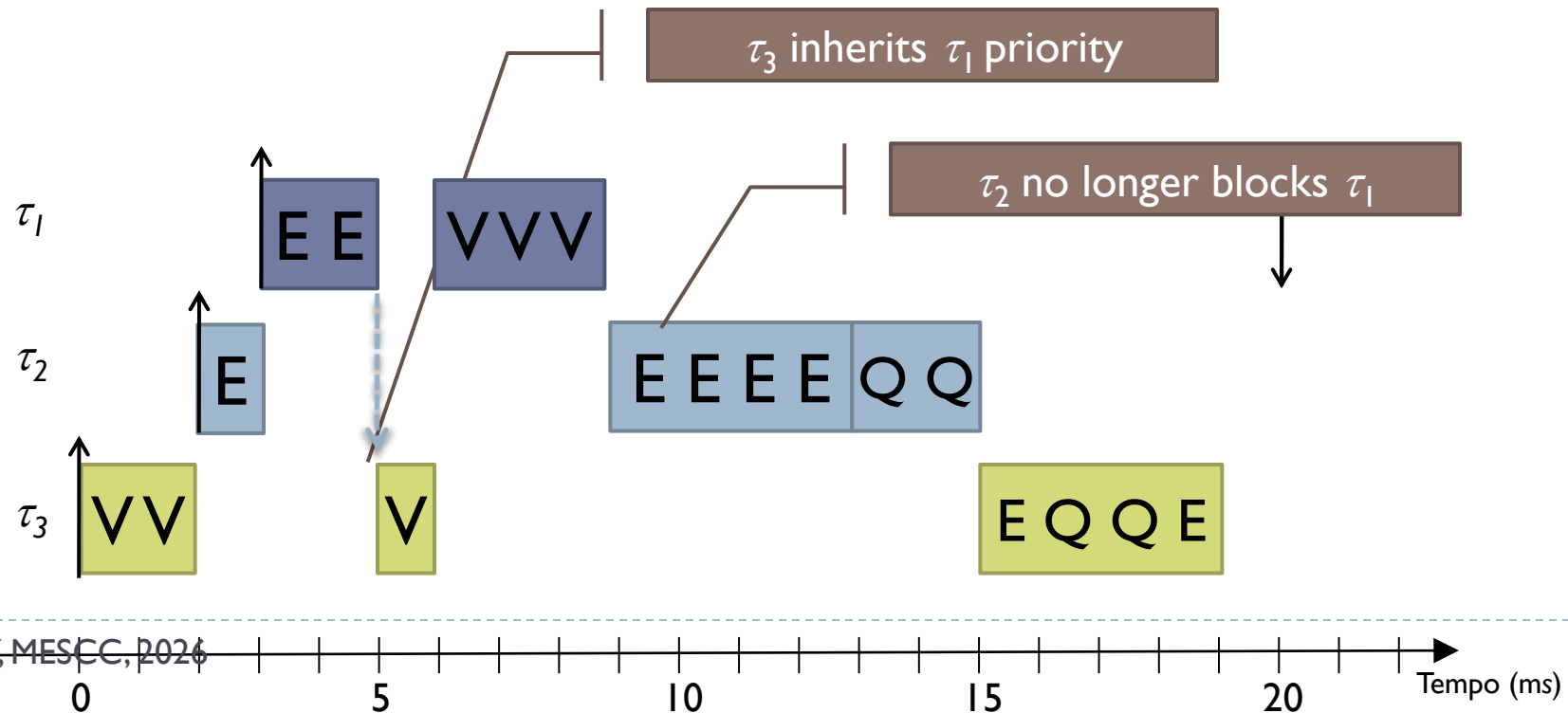
- When a higher priority task is blocked in a shared resource, the lower priority task executing in the resource “inherits” the higher priority while executing in the resource



Resource Sharing

► Priority Inheritance Protocol

- When a higher priority task is blocked in a shared resource, the lower priority task executing in the resource “inherits” the higher priority while executing in the resource



Resource Sharing

- ▶ With the protocol, blocking still occurs when
 - ▶ a lower priority task holds a resource the task needs (direct blocking) or
 - ▶ a lower priority task inherits a higher priority than the task because it holds a resource the higher-priority task needs (push-through blocking)
 - ▶ But a task is only blocked once for each resource it uses
- ▶ General approach to computing blocking times
 - ▶ For all tasks
 - ▶ Examine all tasks with lower priority
 - ▶ Determine the worst-case blocking that it may offer (consider the highest priority that it can inherit)
 - ▶ Examine all resources
 - ▶ Determine the worst-case blocking due to that resource
 - ▶ Consider lower-priority tasks that may inherit a higher priority when they hold the semaphore

Resource Sharing

- ▶ **Maximum blocking time**

- ▶ If all critical sections are of equal length (B)

- ▶ Blocking time of a task

- $B_i = B \times \min(N, R)$

- N is the number of lower priority tasks than i, R is the number of resources used by task i

- ▶ If different lengths

- ▶ Find the maximum length critical section for each resource used by i

- ▶ Add the $\min(N, R)$ biggest critical sections

Resource Sharing

► Maximum blocking of task i

$$B_i = \begin{cases} 0, & i \text{ is the lowest priority task} \\ \sum_{k=1}^K (use(k,i) \times C_k), & \text{other cases} \end{cases}$$

Where ***use(k,i)*** is one if resource k is used by a task with higher or equal priority than task i (or task i)

AND

resource k is also used by at least one lower priority task than task i

If not, use(k,i) is zero

Note that this means that there could be several blocking periods (one per resource)

Resource Sharing

▶ Priority Inheritance Protocol

- ▶ Relatively easy to implement
 - ▶ One additional field for the task information, the inherited priority
 - ▶ Transparent to the programmer – Each task only uses local information
- ▶ Suffers from chain blocking and does not prevent deadlocks

▶ The Priority Ceiling protocol is an extension of priority inheritance

- ▶ Additional rule about access to free resources, inserted to guarantee that all required resources are free
- ▶ For each resource a priority ceiling is defined, which equals the priority of the maximum priority task that uses it
- ▶ A task can only take a resource if the resource is free and if the task priority is greater than the ceilings of all resources currently taken
- ▶ Due to its complexity, the immediate variant is usually preferred

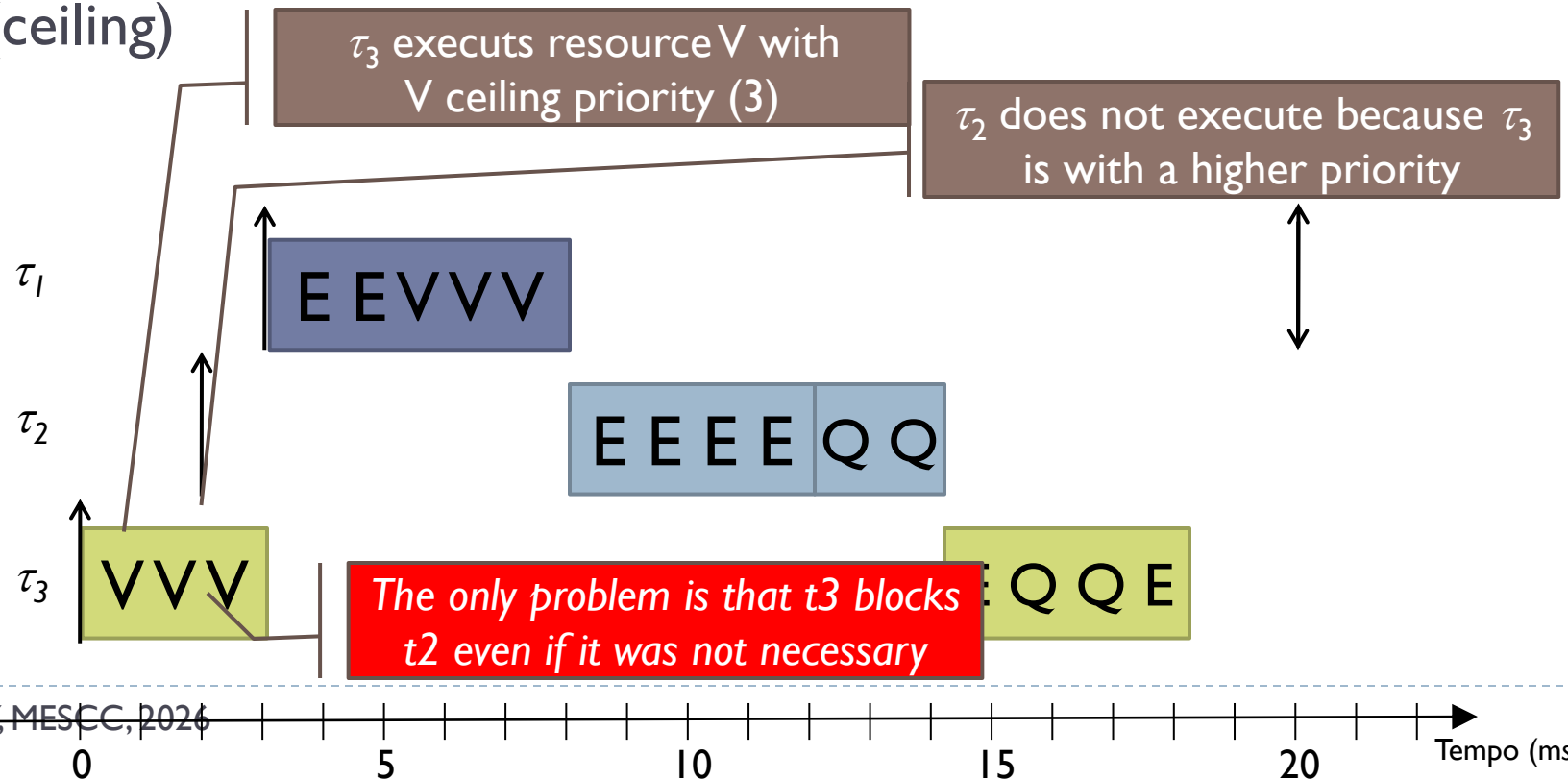
Resource Sharing

► Immediate Priority Ceiling Protocol

- Resources are given a priority which is equal to the highest priority of the tasks that can use the resource (ceiling) – this is determined before execution
- When executing in the resource tasks immediately execute with the priority of the resource (ceiling)

Similar to normal ceiling protocol, it prevents deadlocks

And additionally, in single processor, no locks are required



Resource Sharing

- ▶ With IPCP, it is possible to know the maximum blocking

$$B_i = \begin{cases} 0, & \text{case } i \text{ is the lowest priority task} \\ \max_{k=1}^K \{use(k,i) \times C_k\}, & \text{other tasks} \end{cases}$$

Where ***use(k,i)*** is one if resource *k* is used by a task with higher or equal priority than task *i* (or task *i*)

AND

resource *k* is also used by at least one lower priority task than task *i*

If not, *use(k,i)* is zero

Note that now there is a max function, not a sum: only one blocking period, so worst-case blocking is much less than PIP.

However, resources are always executed with ceiling, so interference on medium priority tasks is higher

Resource Sharing

- ▶ **Schedulability tests**
 - ▶ Utilisation and response time tests can be adapted for the case where tasks share resources
 - ▶ Tests are the same for
 - ▶ Priority Inheritance
 - ▶ Priority Ceiling
 - ▶ Immediate Ceiling
 - ▶ Only the blocking term B_i calculation is different (as previously seen)

Resource Sharing

► Utilization based tests

- With resource sharing, utilization-based tests can be adapted to consider the maximum blocking cost

- But utilization needs to be calculated per task

$$U_i = \sum_{k=1}^i \frac{C_k}{T_k} + \frac{B_i}{T_i} < i(2^{\frac{1}{i}} - 1)$$

- where tasks are (decreasing) ordered per priority ($k = 1$ to i are all tasks with higher or equal priority than i) and B_i refers to the maximum blocking suffered by task i
- Obviously, there is pessimism here (sufficient but not necessary test), and only works for $D_i < T_i$

Resource Sharing

▶ Response-time analysis

- ▶ When there is blocking (due to sharing resources)

$$R_i^{n+1} = C_i + B_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i^n}{T_j} \right\rceil * C_j$$

A blocking term needs to be added to **pre-emptive** Response-Time Analysis

Note 1: there is no problem with non-preemptive, as there is no blocking)

$$R_i^0 = C_i + B_i$$

Note 2: Contrarily to non-preemptive equation, in this case blocking, interference and execution are in the same iterative process.

▶ Determining B_i is per task

- ▶ Dependent of the actual protocol used, but in any case, as we have seen we need to determine which resources can be used simultaneously by tasks with lower priority than t_i and tasks with equal or higher priority than t_i (including t_i itself)

Resource Sharing

► PIP Example:

	<i>C</i> (ms)	<i>T=D</i> (ms)	<i>Priority</i>	<i>Arrival instant (ms)</i>	<i>Execution sequence</i>
τ_1	5	17	3	-	EEVWV
τ_2	5	15	2	-	EEEEQQ
τ_3	7	35	1	-	VVVEQQE

$$C_Q = 2; C_V = 3$$

$$B_1 = use(Q,1) \times C_Q + use(V,1) \times C_V = 0 \times 2 + 1 \times 3 = 3$$

$$B_2 = use(Q,2) \times C_Q + use(V,2) \times C_V = 1 \times 2 + 1 \times 3 = 5$$

$$B_3 = 0$$

Resource Sharing

► IPCP Example:

	<i>C</i> (ms)	<i>T=D</i> (ms)	<i>Priority</i>	<i>Arrival instant (ms)</i>	<i>Execution sequence</i>
τ_1	5	17	3	-	EEVVV
τ_2	5	15	2	-	EEEEQQ
τ_3	7	35	1	-	VVVEQQE

$$C_Q = 2; C_V = 3$$

$$B_1 = \max \{ use(Q, 1) \times C_Q, use(V, 1) \times C_V \} = \max \{ 0 \times 2, 1 \times 3 \} = 3$$

$$B_2 = \max \{ use(Q, 2) \times C_Q, use(V, 2) \times C_V \} = \max \{ 1 \times 2, 1 \times 3 \} = 3$$

$$B_3 = 0$$

Resource Sharing

- ▶ Exercise - consider the following taskset:

	<i>C</i> <i>(ms)</i>	<i>T=D</i> <i>(ms)</i>	<i>Priority</i>	<i>Resource A</i> <i>(ms)</i>	<i>Resource B</i> <i>(ms)</i>	<i>Resource C</i> <i>(ms)</i>	<i>Execution</i> <i>sequence</i>
τ_1	9	30	3	3	0	3	EEAAAECCC
τ_2	8	35	2	3	4	0	EAAABBBB
τ_3	12	50	1	4	3	3	AAAABBBCCCEE

- ▶ Compute the maximum blocking time B_i for each task, using:
 - ▶ the Priority Inheritance Protocol
 - ▶ the Immediate Priority Ceiling Protocol

Resource Sharing

- ▶ Exercise - consider the following taskset:

	<i>C</i> <i>(ms)</i>	<i>T=D</i> <i>(ms)</i>	<i>Priority</i>	<i>Resource A</i> <i>(ms)</i>	<i>Resource B</i> <i>(ms)</i>	<i>Resource C</i> <i>(ms)</i>	<i>Execution</i> <i>sequence</i>
τ_1	9	30	3	3	0	3	EEAAAECCC
τ_2	8	35	2	3	4	0	EAAABBBB
τ_3	12	50	1	4	3	3	AAAABBBCCCEE

- ▶ Determine schedulability of the taskset, both utilization and response time, with
 - ▶ the Priority Inheritance Protocol
 - ▶ the Immediate Priority Ceiling Protocol

Resource Sharing

- ▶ Exercise - consider the following taskset:

	C (ms)	T=D (ms)	Priority	Arrival instant (ms)	Resource A (ms)	Resource B (ms)	Resource C (ms)	Execution sequence
τ_1	9	30	3	3	3	0	3	EEAAAECCC
τ_2	8	35	2	2	3	4	0	EAAABBBB
τ_3	12	50	1	1	4	3	3	AAAABBBCCCEE

- ▶ Trace the execution of the taskset, with the noted arrival instants, for 30 ms, with
 - ▶ the Priority Inheritance Protocol
 - ▶ the Immediate Priority Ceiling Protocol

Resource Sharing

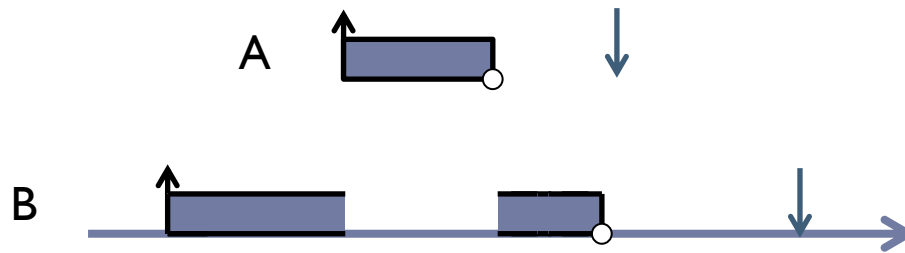
▶ Stack Resource Policy

- ▶ Priority inheritance and priority ceiling protocols are easy to analyze in a fixed-priority setting
- ▶ What about dynamic priority scheduling?
 - ▶ There is no notion of static priority
- ▶ Stack-based resource policy (SRP)
 - ▶ Similar to immediate ceiling priority, but introduces a new concept (preemption level)
 - The capacity of a task to cause preemption on another one (static parameter)
 - ▶ Preemption level
 - if A arrives after B and $\text{priority}(A) > \text{priority}(B)$, then $\text{PreemptionLevel}(A) > \text{PreemptionLevel}(B)$.

Resource Sharing

► Stack Resource Policy

► Preemption level



For EDF, the preemption level can use the relative deadline as the static parameter

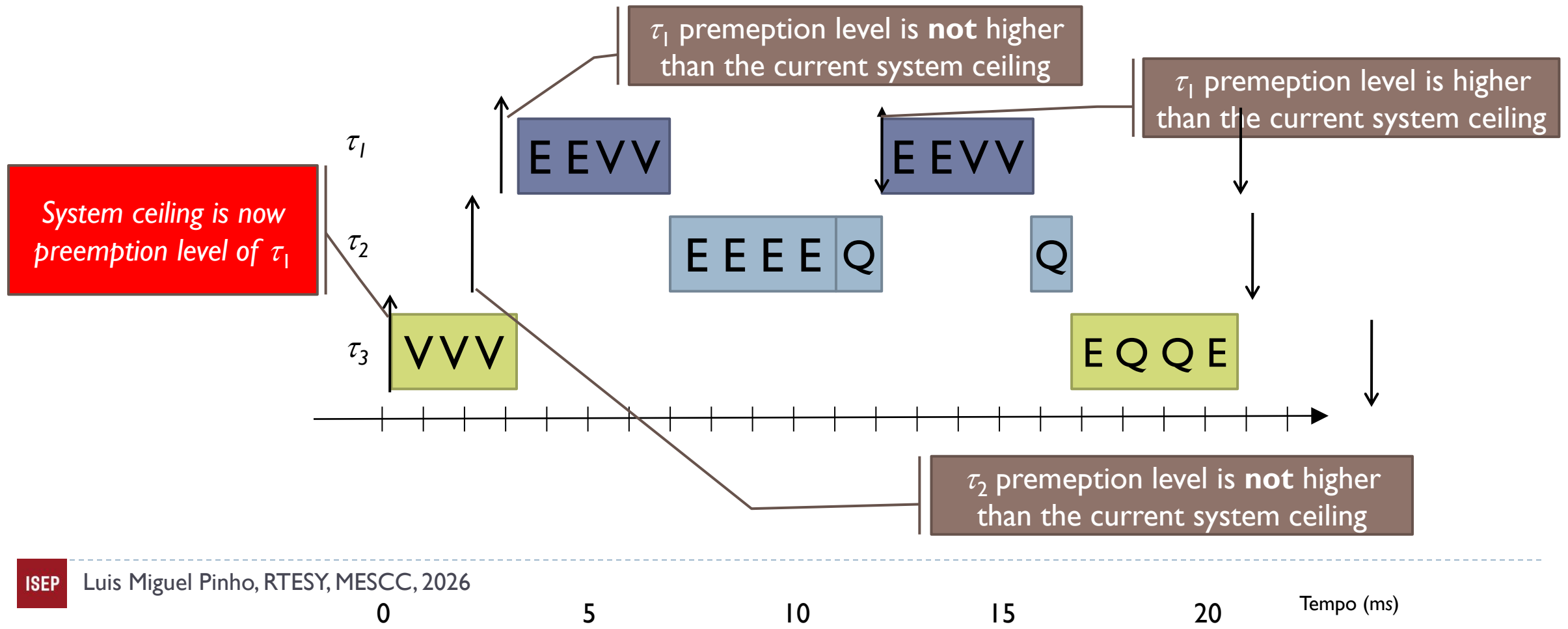
$$D_A < D_B \Rightarrow PL(A) > PL(B)$$

- The resource ceiling for a resource is now the highest preemption level of all tasks that may access the resource
- The system ceiling is the highest resource ceiling among all **currently locked** resources
- A task can preempt another task if it has the highest dynamic priority (earliest absolute deadline for EDF), and its preemption level is higher than the system ceiling

Resource Sharing

Stack Resource Policy

- Example, 3 tasks with relative deadlines: $D_1 < D_2 < D_3 \Rightarrow PL(1) > PL(2) > PL(3)$



Resource Sharing

- ▶ Utilization based tests

- ▶ Similar to fixed-priority

- ▶ Utilization needs to be calculated per task

$$U_i = \sum_{k=1}^i \frac{C_k}{T_k} + \frac{B_i}{T_i} < 1$$

- ▶ where tasks are (decreasing) ordered per preemption level ($k = 1$ to i are all tasks with higher or equal level than i) and B_i refers to the maximum blocking suffered by task i , calculated as in PCP/IPCP (considering preemption levels instead of priorities)
 - ▶ Obviously, there is pessimism here (sufficient but not necessary test), and only works for $D_i < T_i$

Resource Sharing

- ▶ Exercise - consider the following taskset:

	<i>C</i> <i>(ms)</i>	<i>T=D</i> <i>(ms)</i>	<i>Arrival</i> <i>instant (ms)</i>	<i>Resource A</i> <i>(ms)</i>	<i>Resource B</i> <i>(ms)</i>	<i>Resource C</i> <i>(ms)</i>	<i>Execution</i> <i>sequence</i>
τ_1	9	30	3	3	0	3	EEAAAECCC
τ_2	8	35	2	3	4	0	EAAABBBB
τ_3	12	50	1	4	3	3	AAAABBBCCCEE

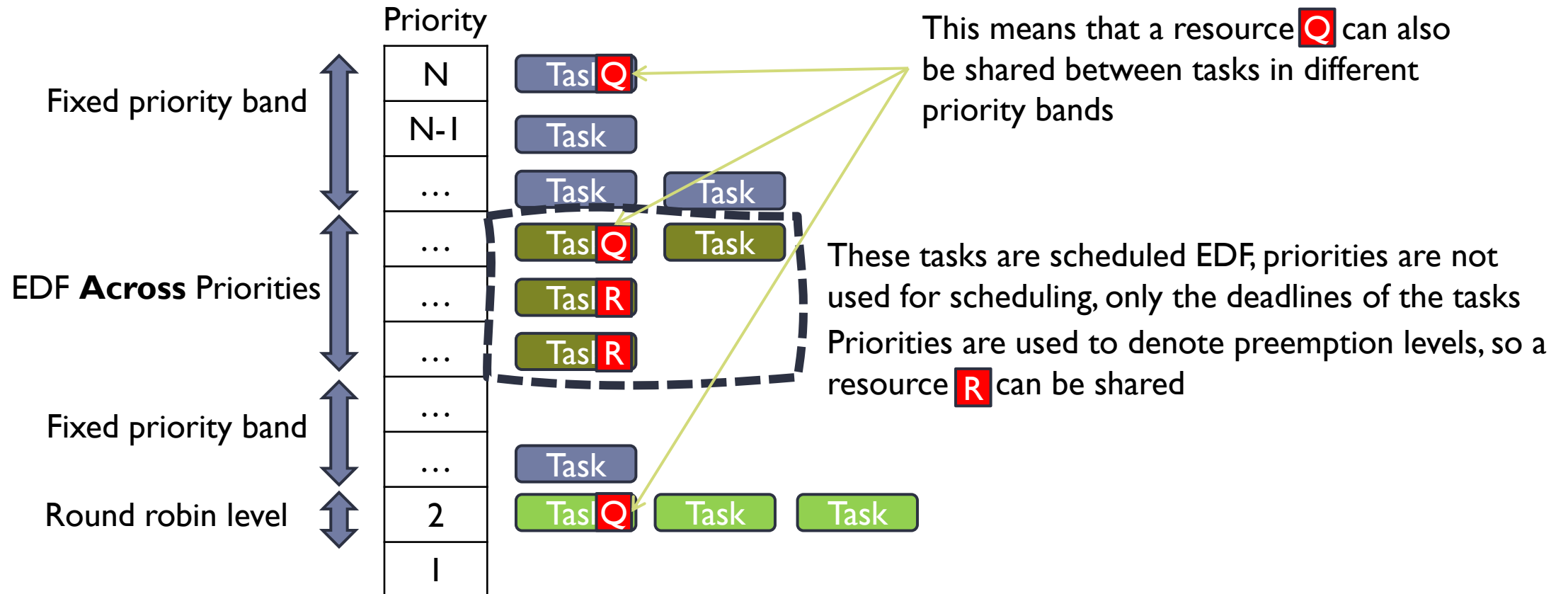
- ▶ Compute the maximum blocking time B_i for each task, using SRP
- ▶ Determine schedulability of the taskset with utilization bounds for SRP
- ▶ Trace the execution of the taskset, with the noted arrival instants, for 30 ms, with SRP

Resource Sharing

▶ Stack Resource Policy

- ▶ Notice that SRP is similar to Immediate Priority Ceiling Protocol (IPCP). However, with no static priority levels, it needs a preemption level
- ▶ SRP (as IPCP) have efficient implementations
 - ▶ Reduce the number of context switches, as blocking is always in the beginning (tasks only start executing when all resources they need are free)
 - ▶ No need for locks in single processor, as IPCP
 - ▶ Also prevents chain blockings and deadlocks as priority ceiling protocols (both PCP and IPCP)
- ▶ More difficult to implement
 - ▶ preemption test more complex, requires computing the system ceiling
- ▶ Important
 - ▶ SRP allows sharing resources between tasks with fixed priority and dynamic priority in hierarchical scheduling

Hierarchical Scheduling in Ada with SRP (deprecated)



Resource Sharing

▶ Deadline Floor protocol

- ▶ A simpler protocol implemented to replace SRP
- ▶ The behaviour is exactly the same as ICPP, but with deadlines
 - ▶ Each resource has a deadline floor, which is the minimum relative deadline of all tasks that can use it
 - ▶ When entering the resource, the deadline of the task is set to
 - Current time + deadline floor
 - The task scheduled by EDF is now the highest priority among those tasks that may use the resource
 - It is therefore guaranteed that it will not be interrupted by any task that can use the resource
 - ▶ When exiting the resource, the task is again at its base deadline
- ▶ Note that it was shown that when jitter occurs, and not considered in the deadline floor, the protocol may fail to provide mutual exclusion, so a real lock should be provided
 - Also in ICPP if the priority ceiling is not well configured, but that is more rare

Resource Sharing

- ▶ Utilization based tests

- ▶ Similar to SRP

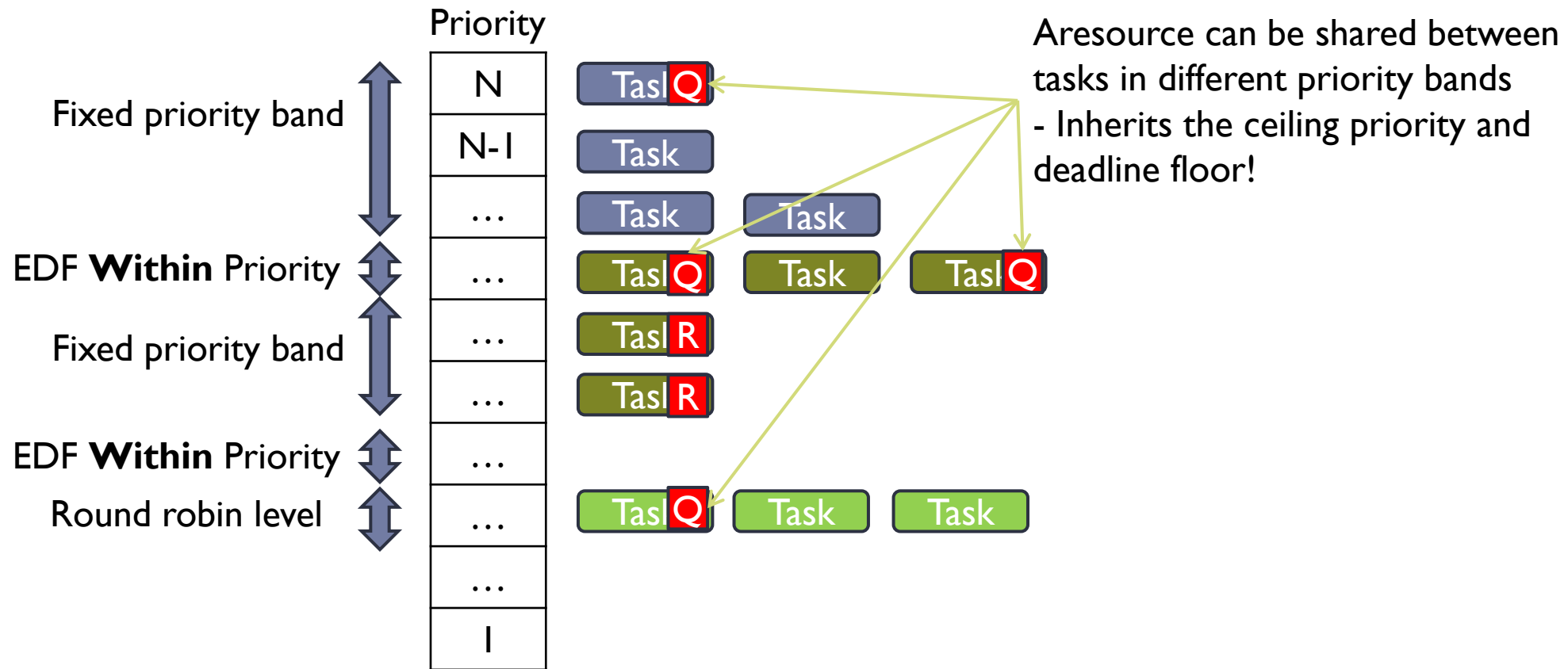
- ▶ Utilization needs to be calculated per task

$$U_i = \sum_{k=1}^n \frac{C_k}{T_k} + \frac{B_i}{T_i} < 1$$

- ▶ But now there are no preemption levels, you have to consider all tasks ($k = 1..n$), and B_i is the maximum blocking suffered by task i , calculated as in PCP/IPCP (considering relative deadline instead of priorities)
 - ▶ Obviously, there is pessimism here (sufficient but not necessary test), and only works for $D_i < T_i$

Hierarchical Scheduling in Ada with Ceiling Locking

► With Deadline Floor protocol



Resource Sharing

▶ Note on Linux

- ▶ Linux implements hierarchical scheduling, but with a different approach
 - ▶ Tasks with real-time scheduling class have precedence over non real-time (Fair Scheduler)
 - ▶ SCHED_DEADLINE (dynamic priorities) is always first
 - ▶ Fixed priority tasks share priorities 1 to 99
 - SCHED_FIFO (FIFO preemptive) and SCHED_RR (round robin)
- ▶ Linux does not implement EDF, but instead a server-based version (CBS like) with SCHED_DEADLINE policy
- ▶ However, for resource sharing, only priority inheritance is supported
- ▶ This means that resource sharing with SCHED_DEADLINE does not solve priority inversion, and schedulability analysis cannot be used
- ▶ There are plans to implement a more advanced algorithm (M-BWI)
 - ▶ Challenge is to implement proxy tasks